

App Development Project Report

App Dev - 2 Project Report

1. Student Details

Name: Rithwik Sriram

Roll Number: 24f3001829

Email: 24f3001829@ds.study.iitm.ac.in

About Me: I am a 3rd Year college student at IIT Madras BS Degree program in Data Science and Applications with a keen interest in web application development and data-driven technologies. I enjoy exploring all tech related avenues to create and build useful applications for all users.

2. Project Details

Project Title: PathFinder V2 - Placement Portal Application

Problem Statement:

To design an advanced web-based platform that streamlines campus recruitment by connecting students with companies, leveraging asynchronous processing for **efficient job drive management and seamless application tracking**.

Approach:

PathFinder was built using Flask as the backend framework with a modular, role-based architecture. The application supports three distinct user roles : **Students, Companies, and Admins**, each with specialized dashboards and functionalities tailored to their needs.

3. AI/LLM Declaration

I used **GitHub Copilot** to assist in writing basic backend boilerplate for Flask (App + API) and SQLite + SQLAlchemy data modeling, representing approximately **5%** of the total project effort. Copilot also aided in scaffolding the initial Vue.js (Core) reactive components, accounting for about 4% of the overall implementation. **ChatGPT (GPT-4)** was utilized for debugging (resolving Celery background task configurations and database relationship issues) and optimizing asynchronous fetch (API Communication), contributing roughly **3%** to the project.

Google Gemini assisted with the CSS transition, providing suggestions to refine the custom CSS layout and UI color scheme, representing about **2%** of the total effort.

The extent of AI/LLM usage is around **12–14%**, primarily limited to code suggestions, documentation formatting, and prompt refinement for the Alpipe API integration.

All core implementation logic including the **authentication system**, **admin approval** workflows, application **database** design and **debugging** were completed manually. The AI features themselves were manually configured.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Vue.js 3	Frontend framework used to build the interactive Single Page Application (SPA)
Celery	Asynchronous task queue for handling scheduled background jobs and CSV exports
Redis	In-memory message broker used by Celery for task routing
Custom Vanilla CSS	Frontend styling and responsive design
Flask-JWT-Extended	Secure, token-based authentication for protecting API endpoints
Flask-Caching	Caching layer to optimize and speed up frequent database queries
SQLite	Lightweight local database for storing user data

5. Database Schema / ER Diagram

Tables:

1. **User** - stores user profile details (id, username, password, role, status)

2. **CompProf** - company profile (id, uid, cname, hr, web)
3. **StudProf** - student profile (id, uid, name, rno, contact, degree, resume)
4. **Drive** - placement drives (id, cid, title, desc, el, deadline, status)
5. **App** - student applications (id, sid, did, dt, status)

Relationships:

- One-to-One → **User** ↔ **CompProf** (uid foreign key)
- One-to-One → **User** ↔ **StudProf** (uid foreign key)
- One-to-Many → **User** → **App** (sid foreign key)
- One-to-Many → **CompProf** → **Drive** (cid foreign key)
- One-to-Many → **Drive** → **App** (did foreign key)



6. API Resource Endpoints

Endpoint	Method	Description
/api/login	POST	Authenticate user credentials and return JWT session token
/api/register/student	POST	Register new student account
/api/register/company	POST	Register new company account
/api/admin/dashboard	GET	Admin dashboard with total statistics
/api/admin/students	GET	View all registered students
/api/admin/companies	GET	View all registered companies
/api/admin/drives	GET	View all placement drives
/api/admin/companies/<id>/status	PUT	Approve/reject/blacklist company accounts
/api/admin/students/<id>/status	PUT	Approve/reject/blacklist student accounts
/api/admin/drives/<id>/status	PUT	Approve/reject placement drives
/api/company/drives	GET/POST	View company drives (GET) or Create new drive (POST)
/api/company/drives/<id>/applications	GET	View applications for a specific drive
/api/company/applications/<id>/status	PUT	Update application status (shortlist/select/reject)
/api/company/applications/<id>/offer_letter	GET	Generate HTML offer letter for a selected application
/api/student/profile	GET/PUT	View and update student profile information
/api/drives	GET	View available placement drives for students

/api/drives/<id>/apply	POST	Submit application for a placement drive
/api/student/applications	GET	View all submitted applications by the student
/api/export/applications	POST	Trigger async Celery task to export applications to CSV

7. Architecture and Features

Architecture Overview:

- **app.py** - main Flask application setup and Celery configuration
- **models.py** - database models using SQLAlchemy
- **routes.py** - centralized RESTful JSON APIs for all roles
- **tasks.py** - asynchronous Celery background tasks
- **celery_worker.py** - entry point for running Celery workers
- **/instance** - SQLite database file
- **/templates** - Jinja2 HTML entry point for the SPA
- **/static** - Vue.js logic (app.js) and frontend assets
- **requirements.txt** - Python dependencies

Implemented Features:

- ❖ **Admin and User Login Functionality**
 - Student and Company registration with role-based access control
 - JWT token-based authentication and secure session management
 - Single pre-existing programmatic Admin account (no admin registration)
- ❖ **Admin Functionalities**
 - Dashboard showing total students, companies, and placement drives
 - Workflows to approve, reject, or blacklist companies and students
 - Workflows to approve or reject placement drives
 - View all student applications with centralized searching
- ❖ **Company Functionalities**
 - Company profile registration and detail management
 - Dashboard showing created drives and applicant counts
 - Create placement drives (only after approval from admin)
 - View, manage, and shortlist student applications
- ❖ **Student Functionalities**
 - Self-registration and secure login

- Dashboard showing all approved placement drives with eligibility filtering
- Submit applications, view status, and maintain placement history
- Edit student profile details and upload resume links
- ❖ **Performance and Caching**
 - Flask-Caching implemented on API endpoints to improve performance
 - Configured cache expiry and optimized response times
- ❖ **Database & Persistence**
 - SQLite + SQLAlchemy
 - Cascade relationships, unique constraints
 - DateTime tracking for applications and deadlines

Additional Features:

- **Asynchronous Backend Jobs**
 - Scheduled daily reminders for upcoming application deadlines
 - Scheduled monthly HTML activity reports generated for the Admin
 - User-triggered asynchronous batch job to export application history as CSV
 - **Other Core Functionalities**
 - Eligibility validation and prevention of multiple applications to the same drive
 - Complete placement history maintenance and dynamic status updates
 - **Optional Enhancements**
 - Custom Dummy Offer Letter Generator for companies to issue HTML offer letters
-

8. Video Presentation

Drive Link:  **App Dev - 2 Project Video Presentation.mp4**
